

```
    return Ok(result);  
}
```

2. Match `dataField` Property Names Exactly

DevExpress sends filter criteria using the exact case-sensitive property name configured on `<dx-column dataField="...">`.

If your `dx-column` specifies `dataField="itemName"`, but EF Core expects `Name`, or vice versa, the search filter will fail or produce SQL generation errors.

Ensure `dataField` in Angular matches your C# model property names (or property casing after JSON serialization):

HTML



```
<dx-data-grid [dataSource]="dataSource" [showBorders]="true">  
  <dxo-search-panel [visible]="true" placeholder="Search items..."></dxo-search-panel>  
  
  <dx-column dataField="id" caption="ID"></dx-column>  
  <dx-column dataField="name" caption="Item Name"></dx-column>  
  <dx-column dataField="price" caption="Price ($)"></dx-column>  
</dx-data-grid>
```

3. Check for Nulls in Database Columns

If searching or filtering crashes the backend, EF Core might be throwing an exception when performing string evaluation (e.g., `Contains`) on database columns containing `NULL` values.

Ensure string columns in your C# entity are initialized or handled, or make sure DevExpress `remoteOperations` is configured properly.

4. How to Debug

1. Open **F12 Developer Tools** in your browser → **Network** tab.
2. Type something into the Search Panel.

3. Look at the outgoing GET request URL:
`http://localhost:7000/api/items?filter=["name","contains","Laptop"]`
4. If the server returns a 500 Internal Server Error or 400 Bad Request , click the request and check the **Response** tab to see the exact C# exception.

i need to remove the search in all columns

To remove the per-column search/filter row from all columns while keeping the grid clean, update your `<dx-data-grid>` configuration in `app.component.html` .

Solution 1: Global Removal via `<dxo-filter-row>`

Add or update the `<dxo-filter-row>` configuration tag with `[visible]="false"` inside your grid declaration:

HTML



```
<dx-data-grid
  [dataSource]="dataSource"
  [showBorders]="true">

  <!-- Disables the search/filter row across all columns -->
  <dxo-filter-row [visible]="false"></dxo-filter-row>

  <!-- Keep global search panel active (optional) -->
  <dxo-search-panel [visible]="true" placeholder="Search items..."></dxo-search-

  <dx-column dataField="id" caption="ID"></dx-column>
  <dx-column dataField="name" caption="Item Name"></dx-column>
  <dx-column dataField="price" caption="Price ($)"></dx-column>
</dx-data-grid>
```

Solution 2: Disable Column Filtering Individually

If you want to keep the filter row visible for the grid as a whole, but disable search capability on specific individual columns, set `[allowFiltering]="false"` on those specific `<dx-column>` tags:

HTML



```
<dx-column dataField="id" caption="ID" [allowFiltering]="false"></dx-column>
<dx-column dataField="name" caption="Item Name"></dx-column>
<dx-column dataField="price" caption="Price ($)" [allowFiltering]="false"></dx-column>
```

```
<div style="padding: 30px;">
```

```
<h2>Simple Inventory Assessment Demo</h2>
```



Here is a refined version of your HTML template with best practices applied:

HTML



```
<div style="padding: 30px;">

  <h2>Simple Inventory Assessment Demo</h2>

  <dx-data-grid
    [dataSource]="dataSource"
    [showBorders]="true"
    [columnAutoWidth]="true"
    [remoteOperations]="true"
    [rowAlternationEnabled]="true">

    <!-- Global Search Bar -->
    <dxo-search-panel
      [visible]="true">
```

```

        [width]="240"
        placeholder="Search items...">
</dxo-search-panel>

<!-- Paging and Pager Options -->
<dxo-paging [pageSize]="5"></dxo-paging>
<dxo-pager
    [showPageSizeSelector]="true"
    [allowedPageSizes]="[5, 10, 20]"
    [showInfo]="true">
</dxo-pager>

<!-- Inline Row Editing Configuration -->
<dxo-editing
    mode="row"
    [allowAdding]="true"
    [allowUpdating]="true"
    [allowDeleting]="true"
    [useIcons]="true">
</dxo-editing>

<!-- Column Definitions -->
<dx-column
    dataField="id"
    caption="ID"
    [allowEditing]="false"
    [width]="80"
    alignment="center">
</dx-column>

<dx-column
    dataField="name"
    caption="Item Name">
    <dx-validation-rule type="required" message="Item Name is required"></dx-validation-rule>
</dx-column>

<dx-column
    dataField="price"
    caption="Price"
    dataType="number"
    format="$#,##0.00"
    alignment="right">
    <dx-validation-rule type="required" message="Price is required"></dx-validation-rule>
    <dx-validation-rule type="range" [min]="0" message="Price must be non-negative"></dx-validation-rule>
</dx-column>

```

```
</dx-data-grid>
```

```
</div>
```

Key Improvements Made

Removed `<dxo-filter-row>` : Completely eliminates the per-column search/filter boxes and magnifying glass icons under each header.

Removed `[keyExpr]='Id'` **from** `<dx-data-grid>` : Key configuration is handled by `createStore({ key: 'id' })` in your TypeScript component. Setting it on both can cause key mismatch conflicts.

Added Validation Rules (`<dx-validation-rule>`): Prevents saving empty or negative values during insert and edit operations, resolving backend model validation errors (`400 Bad Request`).

Added Icon Actions (`[useIcons]="true"`): Replaces text buttons ("Edit", "Delete") with cleaner DevExtreme icons.

Enhanced Pager Controls (`<dxo-pager>`): Adds page size selector options (`5` , `10` , `20`) and item count indicators.

Applied Currency Formatting (`format="$#,##0.00"`): Ensures values display consistently formatted as currency.

ok, i have done my project, Its ok everything..

And i want to move these project into public github, pls tell me the procedure to do

✓

Congratulations on finishing the project!

Here is the step-by-step procedure to push your full-stack Angular and ASP.NET Core project cleanly to a public GitHub repository without exposing sensitive files, connection strings, or temporary build files.

Step 1: Clean Up Sensitive Data and Secrets

Before going public, ensure no private data is committed:

ASP.NET Core (`appsettings.json`): Remove actual database passwords, secret keys, or connection string credentials. Replace them with placeholder values:

JSON



```
"ConnectionStrings": {  
  "DefaultConnection": "Server=localhost;Database=InventoryDb;User Id=YOUR_  
}
```

Angular (`environment.ts`): Check that API base URLs do not contain internal credentials or private tokens.

Step 2: Ensure `.gitignore` Files Are Present

Make sure build artifacts, `node_modules`, and local cache files are excluded from Git tracking.

Root / Frontend `.gitignore`

Ensure your Angular folder (or root directory) has a `.gitignore` containing:

Code snippet



```
# Node & Angular  
node_modules/  
.angular/  
dist/  
.vscode/  
.idea/  
  
# Environment local overrides  
.env
```

Backend .gitignore

In your C# / ASP.NET Core project root, ensure you exclude compiled outputs:

Code snippet



```
# Build outputs
bin/
obj/

# User-specific files
*.user
*.userosscache
*.suo
.vs/
appsettings.Development.json
```

Step 3: Create a New Repository on GitHub

1. Go to [GitHub](#) and click the + icon in the top right → **New repository**.
2. **Repository Name:** Enter a name (e.g., `inventory-management-demo`).
3. **Visibility:** Select **Public**.
4. **Initialize Options:** Leave "Add a README file", ".gitignore", and "License" **unchecked** (since you already have code locally).
5. Click **Create repository**.

Step 4: Initialize Git and Push Your Code

Open your terminal in the root folder of your project (containing both frontend and backend, or your main workspace folder) and run:

Bash



```
# 1. Initialize Git repository locally (if not already initialized)
git init
```

```
# 2. Add all files to staging
git add .

# 3. Create your first commit
git commit -m "Initial commit: Angular frontend with ASP.NET Core DevExtreme API"

# 4. Rename default branch to main
git branch -M main

# 5. Connect your local repository to GitHub (replace URL with your repository URL)
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY_NAME.git

# 6. Push your code to GitHub
git push -u origin main
```

Step 5: Add a Clean README.md File

To make your repository look professional, create a `README.md` file in the root directory explaining how to run your project:

Markdown



```
# Inventory Assessment Demo
```

```
A full-stack Web application built with Angular and ASP.NET Core, featuring DevExtreme
```

```
## Tech Stack
```

```
* **Frontend:** Angular (Standalone Components), DevExtreme Angular DataGrid, Zone.js
* **Backend:** ASP.NET Core Web API, Entity Framework Core, DevExtreme.AspNet.Data
* **Database:** SQL Server / PostgreSQL
```

```
## Getting Started
```

```
### Prerequisites
```

```
* Node.js (v18+)
* .NET SDK (v8.0 / v9.0)
* SQL Server or preferred database instance
```

```
### Backend Setup
```

```
1. Navigate to backend directory: `cd backend`
```


2. Update connection string in `'appsettings.json'`
3. Apply database migrations: `'dotnet ef database update'`
4. Run the API: `'dotnet run'` (Runs on `'http://localhost:7000'`)

Frontend Setup

1. Navigate to frontend directory: `'cd frontend'`
2. Install dependencies: `'npm install'`
3. Run dev server: `'ng serve'` (Runs on `'http://localhost:4200'`)

After adding or updating `README.md` , push the update:

Bash



```
git add README.md
git commit -m "docs: add project setup README"
git push
```

frontend files are missing

If the frontend files are missing from your GitHub repository after pushing, it usually means your `.gitignore` **file in the root directory is accidentally ignoring the frontend folder**, or the frontend folder was added as a **nested Git repository (git submodule)**.

Here is how to check and fix both issues.

Cause 1: Nesting Git Repositories (Most Common)

If your Angular project was created using `ng new frontend` , Angular automatically initializes its own `.git` folder inside that subfolder. When you run `git add .` from the root, Git ignores nested `.git` folders instead of tracking their contents.

Fix:

1. Delete the hidden `.git` folder inside your frontend directory:

Windows (PowerShell):

PowerShell



```
Remove-Item -Recurse -Force .\frontend\.git
```

Mac / Linux:

Bash



```
rm -rf frontend/.git
```

2. Clear Git's index cache for the frontend folder:

Bash



```
git rm --cached -r frontend
```

3. Stage and commit all frontend files:

Bash



```
git add .  
git commit -m "fix: include frontend project files"  
git push
```

Cause 2: Overly Broad `.gitignore` File

If your root `.gitignore` contains a generic pattern like `dist/`, `build/`, or an unspecific folder match, Git might be ignoring your entire frontend directory.

Fix:

1. Open your root `.gitignore` file.
2. Ensure you are ignoring build outputs specifically inside subdirectories rather than ignoring whole folder trees:

Code snippet



```
# Correct specific ignores
frontend/dist/
frontend/.angular/
frontend/node_modules/
```

3. Test if Git recognizes the files:

Bash



```
git status
```

(You should now see all Angular files listed under "Untracked files" or "Changes to be committed".)

Cause 3: Checking Tracked Files

To verify what Git is tracking locally before pushing:

Bash



```
git ls-files frontend/
```

If this command outputs empty, repeat **Cause 1** steps above. Once you see your `.ts`, `.html`, and `package.json` files listed, push to GitHub:

Bash



```
git add .
git commit -m "docs: add frontend codebase"
git push
```

```
# Build outputs
bin/
obj/
```



Your current split setup works, but having `bin/` , `obj/` , `.vs/` , and `*.user` inside the **frontend** `.gitignore` doesn't do anything because those are C# / .NET build artifacts.

For a single root repository containing both `frontend/` and `backend/` folders, keep all ignore rules in **one** `.gitignore` **file located at the very root** of your repository.

Recommended Single Root `.gitignore`

Replace your root `.gitignore` file with this unified version:

Code snippet



```
# =====
# Frontend (Angular & Node)
# =====
frontend/node_modules/
frontend/.angular/
frontend/dist/
frontend/.vscode/
frontend/.idea/
frontend/.env
```

```
# =====  
# Backend (.NET / C#)  
# =====  
**/bin/  
**/obj/  
**/.vs/  
*.user  
*.userosscache  
*.suo  
appsettings.Development.json  
appsettings.Production.json  
  
# =====  
# General System Files  
# =====  
.DS_Store  
Thumbs.db
```

Step-by-Step Fix to Push Missing Frontend Files

If you previously tried to push and the frontend files were skipped, follow these terminal steps from your repository root folder:

1. Remove Any Sub-Git Folder in Frontend

If Angular initialized its own Git repository inside `frontend/`, Git will treat it as a submodule and skip its contents:

Windows (PowerShell):

PowerShell



```
Remove-Item -Recurse -Force .\frontend\.git
```

Mac / Linux:

Bash



```
rm -rf frontend/.git
```

2. Reset Git Cache and Stage Files

Tell Git to untrack the cached folder reference and re-scan everything:

Bash



```
# Clear cached sub-module tracking
git rm -r --cached frontend

# Stage all files according to the new root .gitignore
git add .

# Verify frontend files are now staged
git status
```

3. Commit and Push

Once you see your Angular `.ts` , `.html` ,and `package.json` files listed under green staged changes:

Bash



```
git commit -m "fix: add full frontend codebase to repository"
git push
```

frontend was created using git, so it was not committed in repo, pls do the needful to move the files into repo

To convert the frontend sub-repository into a standard folder inside your root Git repository, run these commands in your project's root terminal:

1. Remove the Embedded `.git` Folder from `frontend`

This removes the independent Git repository tracking inside the Angular directory.

Windows (PowerShell):

PowerShell



```
Remove-Item -Recurse -Force .\frontend\.git
```

Command Prompt (cmd):

DOS



```
rmdir /s /q frontend\.git
```

Mac / Linux:

Bash



```
rm -rf frontend/.git
```

2. Clear Git's Cache for the `frontend` Submodule

Clear the cached gitlink reference in the parent repository:

Bash

